

The Three Tiers, Explained

What each tier does, why it exists, and who builds it

Read this first

EvidenceIQ is not three products. It is **one experiment run three times**. Each tier adds exactly one capability, so the difference between them measures the value of that capability. That is the whole contribution of the project.

1. The problem

Ask any standard LLM what our revenue was in 2011 and it will answer with a confident figure and a pound sign. That figure never touched our database — it came from the model's parameters. It looks exactly like a correct answer.

That is what makes it dangerous. A system that says "I don't know" is annoying but safe. A system that says **£8,400,000** when the truth is **£9,809,614.01** is acted upon, and nobody catches it for months.

EvidenceIQ answers business questions by actually running the query, and then a separate piece of code checks every number in the written answer against what the query returned. Anything that cannot be traced is blocked before the user sees it.

2. The shape all three tiers share

Every tier is the same function signature:

```
ask(question: str) -> Answer
```

Answer always contains the same fields: findings, claims[], kpis, fields_used, filters_used, chart_spec, limitations, insufficient_data, tool_calls[].

What changes between tiers is how many stages run before that Answer comes back. Keeping the shape identical is deliberate: it is what lets one evaluation harness score all three fairly.

Tier	What it adds	What it still gets wrong
Tier 1 Direct LLM	nothing — the question and a schema description	every number is invented
Tier 2 Tool agent	SQL / Python / chart tools, and a tool log	the numbers are real, but the written answer is never checked against them
Tier 3 Verified agent	a verification pass and a retry loop	slower and more expensive — the trade-off we report honestly

3. The worked example used throughout

Question **q11** from our benchmark:

"Did November 2011 beat October 2011 on revenue, and by how much?"

True answer, computed by us in SQL: **£1,456,145.80 vs £1,069,368.23, up 36.17%**. Both months had 26 trading days, so the comparison is like-for-like.

This question is useful because it exercises everything: a SQL query, a derived calculation, a comparison that could be misreported, and a trading-day caveat.

4. Tier 1 — Direct LLM

Stages: one. question → LLM → Answer

What it gets

- The question
- A description of the schema: column names, the 2009–2011 date range, and an explicit statement that there is no cost, profit or competitor data

What it does **not** get is any way to run a query. No database connection exists in this code path at all.

What happens on q11

The model has seen retail datasets in training. It knows November is peak season for a gift wholesaler. So it produces something structurally plausible and entirely invented:

```
"November 2011 outperformed October, with revenue of approximately
GBP 1,200,000 versus 950,000, an increase of around 26%."
```

Not random — *plausible*. That is precisely what makes it unsafe.

Why we build it at all

It is the control group. Without it we can only say "our system is good". With it we can say "our system is *this much* better than the obvious alternative".

Important wording for the report

Tier 1's evidence coverage is 0% and its unsupported-claim rate is 100% **by construction, not by measurement**. No query was executed, so nothing can be traced. Claiming we "measured" 0% evidence coverage would be sloppy.

Tier 1 wins on latency and cost

One call, no tools, fewest tokens. Report that plainly — it is the trade-off that makes the Tier 3 result meaningful rather than a free lunch.

Application in our repo

Item	Detail
Notebook	notebooks/03_baseline_llm.ipynb
Model	Gemma via Ollama — local, free, no API key
Output	eval/results/baseline_tier1.csv and baseline_tier1_summary.csv
Owner	run once, then it is a fixed reference point

5. Tier 2 — Tool-using agent

Stages: four. question → plan → execute tools → draft Answer

Stage 1 — Plan

The model breaks the question into an ordered plan:

- get revenue for 2011-11 and 2011-10
- compute the percentage change
- check the trading-day counts are comparable
- draw a comparison chart

The plan is stored and displayed — "structured planning and tool-selection workflow" is a required deliverable.

Stage 2 — Execute tools

Tool	Purpose
run_sql	one read-only SELECT against evidenceiq.duckdb
run_python	calculations awkward in SQL — percentage changes, ratios, shares
make_chart	the model must choose the chart type; this is a scored metric

For q11 the model writes real SQL, which returns real rows:

```
SELECT invoice_month, net_revenue, trading_days
FROM dim_month
WHERE invoice_month IN ('2011-10', '2011-11');

-- 2011-11 | 1456145.80 | 26
-- 2011-10 | 1069368.23 | 26
```

The tool log — the most important object in the system

Every call appends an entry:

```
ToolCall(
  index = 0,
  tool = "run_sql",
  code = "SELECT invoice_month, net_revenue ...",
  ok = True,
  rows = [{"invoice_month": "2011-11", "net_revenue": 1456145.80, ...},
          {"invoice_month": "2011-10", "net_revenue": 1069368.23, ...}]
)
```

Understand this deeply

The tool log is the **only** source of truth about what actually happened. Verification is not "ask the model if it is sure". It is "does this number exist in this log — yes or no". Everything in Tier 3 depends on this object being complete and honest.

Stage 3 — Structured answer

The model returns JSON, not prose. Claims come out as objects:

```
{
  "findings": "November 2011 revenue was higher than October ...",
  "claims": [
    {
      "text": "November revenue was GBP 1,456,145.80",
      "value": 1456145.80,
      "unit": "GBP",
      "source_tool_call": 0
    },
    {
      "text": "an increase of 36.17%",
      "value": 36.17,
      "unit": "%",
      "source_tool_call": 1,
      "derivation": "pct_change",
      "derived_from": [1456145.80, 1069368.23]
    }
  ]
}
```

This is not a stylistic choice. Extracting numbers from a paragraph with regular expressions is unreliable, and if claim extraction is unreliable then verification is worthless. **The structure is what makes Tier 3 possible.**

Application in our repo

Item	Detail
Owner	Zayed
Deliverable	ask_tier2(question) — the single function Tier 3 calls
Key risk	the model forgets NOT is_cancellation and silently returns the wrong revenue. Put the rule in the prompt <i>and</i> in the worked examples
Few-shot source	docs/kpi_definitions.md — the KPI formulas teach the rules
Owns metrics	execution success rate, tool-selection accuracy

6. Tier 3 — Verified agent

Stages: six. question → plan → tools → draft → **verify** → (retry) → Answer

Tier 3 does not rewrite the agent. It **wraps** it. The orchestration is thin; all the intelligence sits in the verifier.

```
def tier3(question, max_retries=3):
    log, feedback = ToolLog(), ""
    for attempt in range(max_retries + 1):
        answer = run_tier2_pass(question, log, extra=feedback)
        answer = verify(answer, log)
        feedback = critique(answer)
        if not feedback:
            break
    return answer
```

Check 1 — Support

For every claim with a numeric value, search all successful tool results for a matching number within 1%.

- **1456145.80** — found in call 0 → **supported**, evidence attached
- **36.17** — found in call 1 (the Python calculation) → **supported**
- **1,400,000** — rounded in the model's head, never computed → **unsupported**

This is the hallucination detector. It is a *search*, not a judgement.

Check 2 — Recompute

For claims carrying a derivation, re-derive the value independently:

```
new, old = claim.derived_from      # 1456145.80, 1069368.23
expected = (new - old) / old * 100 # 36.17

# model said 12% -> flagged: "stated 12, recomputed 36.17"
```

Why this is a separate check from Check 1

A number can pass support and still be wrong. If some unrelated query happened to return the value 12, the support search finds it and the claim looks fine. Only recomputation catches that the *derivation* is wrong. The two checks fail independently, which is exactly why both exist.

Check 3 — Reconcile

If the answer breaks a change down by country or product, those contributions must sum to the overall change. Shares must sum to roughly 100%. This check is required explicitly by the project brief.

The retry loop

When a check fails we do not simply reject — we write a critique and send it back:

```
"an increase of 12%" -> flagged: recomputation mismatch,
    stated 12, recomputed 36.17.
Only state numbers that a tool call actually returned.
```

Up to three attempts. Anything still failing is **withheld** from the user, not displayed with a warning.

What the user finally sees

November 2011 revenue was GBP 1,456,145.80, compared with
GBP 1,069,368.23 in October - an increase of 36.17%.
Both months had 26 trading days.

```
[OK] GBP 1,456,145.80    show evidence > SELECT invoice_month, ...  
[OK] 36.17%            show evidence > (1456145.80 - 1069368.23)  
                               / 1069368.23 * 100
```

Fields: net_revenue, invoice_month, trading_days
Filters: invoice_month IN ('2011-10', '2011-11')

Seven fields are required in this display: findings, KPI values, chart, the SQL/Python, fields and filters used, verification status per claim, and limitations or insufficient-data warnings.

Application in our repo

Item	Detail
Owner	Student A
Not blocked	build the verifier against hand-written fake tool logs. No database, no LLM, no working SQL needed — start immediately
Tests first	supported number passes; invented number unsupported; bad arithmetic flagged; shares summing to 70% caught
Hard rule	no LLM inside the verifier. A model checking another model's arithmetic inherits the same failure mode
Owns metrics	evidence coverage, unsupported-claim rate, and the evaluation harness

7. The application — where all three tiers become visible

The app is the only part of the project a non-technical person will ever touch. It has **four tabs**, and they are not four views of the same thing — two of them are separate required deliverables in their own right.

Tab	What it is	Why it exists
1	Baseline analytics dashboard	a required deliverable. No AI at all — the human-analyst reference point our agent is compared against
2	Ask	the product itself: question in, verified answer out, with the evidence trail
3	Compare tiers	the same question through Tiers 1, 2 and 3 side by side — this is what sells the project in the demo
4	Evaluation dashboard	a required deliverable. The scoreboard across all three tiers

7.1 Tab 1 — Baseline analytics dashboard (no AI)

Build this **first**. It needs no LLM, no agent and no verifier — only the database, which already exists. It means we have something demoable from week 2 regardless of how the agent work goes.

Contents

- **A KPI strip across the top** — revenue, orders, average order value, active customers, cancellation rate. Use st.metric.
- **Revenue by month** — a line chart from dim_month.
- **Top 10 products** — horizontal bar, from dim_product or a grouped query.
- **Top 10 countries excluding the UK** — horizontal bar. The UK is 83.5% of revenue, so including it flattens everything else.
- **A returns panel** — cancellation rate over time.

Sanity numbers — if the dashboard does not show these, a filter is wrong

KPI (whole dataset)	Expected	Rule
Revenue	20,465,198.39	NOT is_cancellation
Orders	45,330	COUNT(DISTINCT invoice_no)
Average order value	451.47	revenue / orders
Active customers	5,881	customer_id IS NOT NULL
Cancellation rate	18.29%	cancellations / completed orders
Units sold	11,455,906	quantity > 0

Two traps in the dashboard itself

December 2011 is incomplete — 8 trading days, not 26. On the revenue-by-month chart either exclude it or mark it clearly, otherwise the trend line appears to collapse. **Top products must filter is_product AND NOT is_outlier**, or "Manual" and "DOTCOM POSTAGE" appear as best sellers.

7.2 Tab 2 — Ask

A tier selector, a question box, and the answer. The brief requires **seven things** on screen. All seven come straight off the Answer object, so this is display work, not logic.

Required field	Where it comes from	How to show it
Answer and key findings	answer.findings	st.markdown
Supporting KPI values	answer.kpis	st.metric in columns
Relevant visualization	answer.chart_spec	render with plotly
SQL query or Python	tool_call.code	st.code inside the expander
Data fields and filters used	answer.fields_used, answer.filters_used	a caption line under the answer
Verification status	claim.status	icon per claim (see below)
Limitations / insufficient data	answer.limitations, answer.insufficient_data	st.info or st.error banner

7.3 The evidence panel — the most important screen in the project

Every number gets a status icon, and clicking it reveals the exact query that produced it. This single interaction is what the whole project is arguing for, so it deserves more care than anything else in the UI.

```
for claim in answer.claims:
    icon = {"supported": "OK",
           "flagged": "WARN",
           "unsupported": "FAIL"}[claim.status]

    with st.expander(f"{icon} {claim.text}"):
        st.write(f"Status: {claim.status}")
        if claim.failure_reason:
            st.warning(claim.failure_reason)
        if claim.evidence:
            st.code(claim.evidence, language="sql")
            st.dataframe(rows_for(claim.source_tool_call))
```

Status	Icon	What the user should understand
supported	green check	this number came from a query you can inspect
flagged	amber warning	the number exists but the arithmetic or the breakdown did not check out — show the reason
unsupported	not shown	withheld entirely. Display a count ("1 claim withheld") but never the claim itself

Withheld does not mean hidden

Unsupported claims are removed from the answer, but the user should still be told that something was removed. Silently dropping content is its own kind of dishonesty. A small line — *"1 claim could not be verified and was withheld"* — is enough.

Underneath the claims, add a collapsed **full tool log**: every call, its code, whether it succeeded, and the rows it returned. This is what a sceptical reviewer will open at the demo.

7.4 Tab 3 — Compare tiers

Three columns, one per tier, same question. Keep it simple: findings, the headline number, and a claim count with how many were unsupported.

Default the box to a trick question

Pre-fill it with *"What was our profit margin in 2011?"*. Tier 1 will invent a margin. Tier 3 should refuse and explain that there is no cost data. That side-by-side is the single most persuasive screen we can put in front of anyone.

7.5 Tab 4 — Evaluation dashboard

Reads the CSVs the harness writes — no live model calls.

- **Per-tier metric table** from eval/results/summary.csv
- **Unsupported-claim rate by tier** — a bar chart. This is the headline result of the whole project

- **Accuracy by difficulty** — grouped bars for easy / medium / hard
- **Refusal behaviour** — how each tier handled the three impossible questions
- **Latency and cost** — shown plainly, including where Tier 3 loses

7.6 How to build without waiting for anyone

This lane is the least blocked and should never be idle. The trick is to work against a **hard-coded fake Answer** until the real agent lands:

```
FAKE = {
  "findings": "November 2011 revenue was higher than October.",
  "claims": [
    {
      "text": "November revenue was GBP 1,456,145.80",
      "value": 1456145.80,
      "status": "supported",
      "evidence": "SELECT invoice_month, net_revenue FROM dim_month ...",
    },
    {
      "text": "an increase of 12%",
      "value": 12.0,
      "status": "flagged",
      "failure_reason": "stated 12, recomputed 36.17",
    },
  ],
  "kpis": {"Nov revenue": 1456145.80, "Oct revenue": 1069368.23},
  "fields_used": ["net_revenue", "invoice_month"],
  "limitations": None,
  "insufficient_data": False,
}
```

Build the entire Ask tab against that dictionary. When `ask_tier3()` is ready, swap one line. Order of work:
Tab 1 → Tab 4 → Tab 2 → Tab 3.

7.7 Practical Streamlit notes

- **Cache the database reads.** Wrap dashboard queries in `@st.cache_data` or every widget click re-runs every query.
- **Open DuckDB read-only** in the app too, not just in the agent.
- **Never call the model on page load.** Only inside a button press, or the app fires a request on every rerun.
- **Show a spinner** — Gemma running locally takes seconds, and a frozen page looks broken.
- **Keep charts in one helper** that turns a `chart_spec` into a figure, so Tabs 1 and 2 render identically.

Definition of done for this lane

Someone who has never seen the project can open the app, ask a question, read the answer, click any number, and see the SQL that produced it — without anyone explaining anything to them.

8. Working in parallel — nobody waits for anybody

The three lanes look sequential — Tier 2 feeds Tier 3, Tier 3 feeds the app — but they are not. All three can be built at the same time, provided we do one thing first.

The whole trick, in one sentence

Agree the **shape of the data** that passes between us on day one, commit a fake example of it, and then everyone builds against the fake until the real thing arrives. Integration then becomes a one-line swap instead of a week of arguing.

8.1 The contract — do this in the first session, together

Three objects pass between the lanes. Agree them once, write them into `src/schemas.py`, and treat any change afterwards as something that needs telling the others about.

Object	Produced by	Fields
ToolCall	Zayed (Tier 2)	index, tool, code, ok, rows, error
Claim	Zayed, annotated by Student A	text, value, unit, source_tool_call, derivation, derived_from, then status, evidence, failure_reason
Answer	Zayed, consumed by everyone	question, findings, claims[], kpis, fields_used, filters_used, chart_spec, limitations, insufficient_data, tool_calls[]

Zayed commits the file, because Tier 2 is what emits these objects. But it is agreed by all three — Student A verifies them and Student B renders them.

8.2 Commit two fixtures on day one

Put a realistic fake tool log and a fake answer in the repo. This is the single highest-value thirty minutes of the whole project.

<code>eval/fixtures/sample_tool_log.json</code>	<code><- what Tier 2 produces</code>
<code>eval/fixtures/sample_answer.json</code>	<code><- a draft answer with one good claim, one invented claim, one bad calculation</code>

Student A tests the verifier against the fake tool log. Student B renders the fake answer. Zayed makes real Tier 2 output match the fixture. Nobody is waiting, and when the pieces meet they already agree.

8.3 What each person can build today, needing nobody

Person	Start with this — no dependencies at all	Only needs someone else for
Zayed (Tier 2)	run_sql and the tool log, tested with hand-written SQL — no LLM involved. The database already exists. Prove a query runs, gets logged, and comes back as a ToolCall. Add the model afterwards.	nothing. This lane is fully unblocked from minute one.

Person	Start with this — no dependencies at all	Only needs someone else for
Student A (Tier 3)	the verifier and its tests, against the fake tool log. No database, no LLM, no working SQL required. Write the four tests first, then make them pass.	a real ask_tier2 — but only at the very end, to run a live Tier 3. The verifier itself is finished before that.
Student B (App)	Tab 1 (baseline dashboard) and Tab 4 (evaluation) — neither touches the agent at all. Then build Tab 2 against the fake answer.	the results CSVs for Tab 4, which can be faked with three rows until the harness runs.

8.4 Rules that keep the lanes moving

- **Blocked for more than thirty minutes? Stub it.** Write the function signature, return a hard-coded value, add a TODO, and carry on. Then say so in the group chat — do not sit on it quietly.
- **Mock at the seam, not deeper.** Student A fakes the tool log, not the database. Student B fakes the answer, not the verifier. Fake the thing your neighbour owns, nothing further.
- **Ship the ugly version first.** Tier 2 as a single model call that writes one query beats a well-designed reasoning loop that does not exist yet. Improve it once something runs end to end.
- **Never block on a decision.** If it can be reversed later, pick one, write down what you picked, and move. Bring it to the weekly check-in.
- **Push small and often.** A branch that lives for a week is a merge conflict with a countdown timer.

8.5 The two moments we actually have to sync

When	What has to be true
End of week 3	Zayed's ask_tier2 returns a real Answer for the five easy questions. Student A's verifier passes its tests on fixtures. Student B's Tab 1 is live. If Tier 2 is behind, the other two stop their own work and help — everything downstream depends on it.
End of week 5	Tier 3 runs live: real SQL, real verification, real retries, rendered in the app. From here it is testing and polish only.

If you remember one thing

The person most likely to be blocked is whoever waits for something to be *finished*. Nobody needs finished work — they need the **shape** of it. Agree the shape, fake the content, and all three lanes run at full speed.

9. What the comparison proves

Gap	What it measures
Tier 1 → Tier 2	the value of grounding — using real data instead of memory
Tier 2 → Tier 3	the value of verification — checking the narrative against the data

If we had built only Tier 3 we could claim it works, but not say which half did the work. Two gaps, two separate findings. That is the ablation, and it is our contribution.

Report the trade-off honestly

Tier 3 will be slower and more expensive — verification adds a pass and retries add calls. State it plainly: *"verification costs X seconds and Y tokens, and reduces unsupported claims from Z% to near zero."* A quantified trade-off is more credible than a clean win.

10. Rules that apply to every tier

- **Read-only database.** The agent must never write. Writes are rejected before execution.
- **Never claim causation.** Say "the largest contributors were ...", never "revenue fell because ...". We have no data on promotions, pricing or competitors.
- **Refuse rather than estimate.** If the data cannot answer, set `insufficient_data` and explain what is missing.
- **No forecasting.** Two years of one retailer's sales is not a basis for prediction.

Data facts every tier must respect

Fact	Consequence
The two raw sheets overlap by nine days	Handled at build time. Without it December 2010 reads 1,126,445 instead of the true 746,724 — 51% too high.
December 2011 has only 8 trading days	Data stops on the 9th. Never a valid month-over-month comparison. Check <code>dim_month.is_complete_month</code> .
Months have different trading-day counts	March 2011 had 27, April had 21. Revenue "fell 28%" but only 7% per trading day. Always state the day count.
The UK is 83.5% of revenue	"Which country earns most" is a constant, not a question.
Most "returns" are not returns	Fees, manual adjustments and two freak cancelled orders. Naive rate 8%+, real product rate 2.36%.

11. Who builds what

Owner	Tier	Definition of done
Zayed	Tier 2 — SQL generation, execution, tool log	all 12 answerable benchmark questions return real numbers from real queries, and the SQL can be shown for each
Student A	Tier 3 — verification layer, retry loop, evaluation harness	given a fake tool log and a lying answer, the verifier catches an invented number, wrong arithmetic and a breakdown that does not add up — with passing tests
Student B	Application — dashboards and the evidence panel	someone who has never seen the project can ask a question, get an answer, and click through to the SQL behind any number

The one dependency that matters

Student A cannot run a *real* Tier 3 until Zayed's ask_tier2 exists — but the verifier itself can and should be built first against fake logs. Everything else runs in parallel. If Tier 2 is behind at the end of week 3, everyone helps.